Ajustes BE requeridos — App Móvil TTPN (Android)

Fecha: 2026-04-10
Fuente: Análisis de `/app` (Android nativo Java) + `/ttpn_php` (257 archivos PHP)

Resumen ejecutivo

La app móvil es Android nativo (Java) que actualmente se comunica contra un backend PHP hosteado en Heroku.
Base URL actual: `https://ttpnplaystore-a94805096f88.herokuapp.com/`

El Rails API debe reemplazar esos 257 endpoints PHP en rutas RESTful.
Esta doc lista **solo los ajustes necesarios en BE** para que la app funcione.

---

1. AUTENTICACIÓN

Situación actual (PHP)

- La app hace GET a `Gasto_SELECT_USUARIOS.php` — descarga TODOS los usuarios
- Compara el password ingresado con `encrypted_password` **en el cliente** usando jBcrypt
- Guarda `role_id`, `user_id` y token en SharedPreferences

Lo que necesita el Rails API

```
POST /api/v1/auth/login
Body: { email, password }
Response: {
  access_token,
  token_type: "Bearer",
  expires_in: 86400,
  user: { id, email, nombre, role_id, role, sadmin, business_unit_id },
  privileges: { ... }
}
```

Estado actual: ✅ Ya implementado en `api/v1/auth/sessions_controller.rb`

Pendiente: La app usa `role_id` directamente — verificar que el campo `role_id` venga en la respuesta.
Acción: Verificar que la app acepte JWT en header `Authorization: Bearer <token>` en lugar de sesión PHP.

---

2. TRAVEL COUNTS ⚠ CRÍTICO

Qué hace la app

1. El chofer registra un viaje → `Gasto_INSERT_TRAVEL_COUNTS.php`
2. El PHP ejecuta stored procedures `buscar_booking()` y `buscar_booking_id()` para encontrar el `ttpn_booking` correspondiente
3. Si encuentra match, actualiza `ttpn_bookings.travel_count_id`
4. Genera una discrepancy de tipo `pnc` si no encuentra match

Endpoints PHP requeridos → Rails equivalente

PHP endpoint	Rails endpoint necesario	Método
`Gasto_INSERT_TRAVEL_COUNTS.php`	`POST /api/v1/travel_counts`	POST
`Gasto_ACTUALIZAR_TRAVEL_COUNTS.php`	`PUT /api/v1/travel_counts/:id`	PUT
`Gasto_SELECT_TRAVEL_COUNTS_UN_REGISTRO.php`	`GET /api/v1/travel_counts/:id`	GET
`Gasto_ACTUALIZAR_TRAVEL_COUNT_AUTORIZADO.php`	`PUT /api/v1/travel_counts/:id/autorizar`	PUT
`Gasto_ACTUALIZAR_TRAVEL_COUNT_REQUIERE_AUTORIZACION.php`	`PUT /api/v1/travel_counts/:id/requiere_autorizacion`	PUT
`Gasto_ACTUALIZAR_TRAVEL_COUNT_RECHAZADO.php`	`PUT /api/v1/travel_counts/:id/rechazar`	PUT
`Gasto_INSERT_TRAVEL_COUNTS_COMENTARIO.php`	`POST /api/v1/travel_counts` (mismo, campo `comentario`)	POST

PHP endpoint	Rails endpoint necesario	Método
`Gasto_INSERT_TRAVEL_COUNTS_COMENTARIO_POR_ADMIN.php`	`POST /api/v1/travel_counts/:id/comentarios`	POST

Campos que envía la app al crear un travel_count

```
{
  "id": 12345,
  "vehicle_id": 5,
  "employee_id": 10,
  "client_branch_office_id": 3,
  "ttpn_service_type_id": 1,
  "ttpn_foreign_destiny_id": 2,
  "fecha": "2026-04-10",
  "hora": "08:30:00",
  "costo": 250.00,
  "status": true,
  "viaje_encontrado": false,
  "comentario": "opcional"
}
```

⚠ **Importante:** La app genera el `id` en el cliente (fetch max + 1). El Rails API debe ignorar ese `id` y usar autoincrement.

Lógica crítica: Booking matching

El PHP tiene stored procedures que Rails debe replicar. La lógica es:

```
-- Busca un ttpn_booking que coincida con el travel_count en ventana de ±15 min
SELECT id FROM ttpn_bookings
WHERE vehicle_id = :vehicle_id
  AND employee_id = :employee_id
  AND ttpn_service_type_id = :ttpn_service_type_id
  AND fecha = :fecha
  AND hora BETWEEN (:hora - interval '15 minutes') AND (:hora + interval '15 minutes')
  AND status = true
  AND viaje_encontrado = false
LIMIT 1
```

Si encuentra match:

- `travel_counts.viaje_encontrado = true`
- `travel_counts.ttpn_booking_id = <id encontrado>`
- `ttpn_bookings.travel_count_id = <nuevo travel_count_id>`
- `ttpn_bookings.viaje_encontrado = true`

Si NO encuentra match:

- Crear registro en `discrepancies` con `kpi = 'pnc'`

Esta lógica ya existe en `TtpnBooking` como callback `create_actualiza_tc` — pero hay que verificar que también funcione en sentido inverso (desde `travel_count` hacia `booking`).

Campos que responde la app al ver un travel_count

```
{
  "v.clv": "DEV-V001",
  "tc.client_branch_office_id": 3,
  "ttpn_service_type_id": 1,
  "fecha": "2026-04-10",
  "hora": "08:30:00",
  "ttpn_foreign_destiny_id": 2,
  "costo": 250.00,
  "status": true,
  "vehicle_id": 5,
  "employee_id": 10
}
```

Acción requerida: Verificar que `TravelCountsController` existe y expone estos campos + el `clv` del vehículo en el serializer.

---

3. TTPN BOOKINGS

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_INSERT_TTPN_BOOKINGS.php`	`POST /api/v1/ttpn_bookings`	POST
`Gasto_SELECT_TTPN_BOOKINGS.php`	`GET /api/v1/ttpn_bookings?coo_travel_request_id=X`	GET
`Gasto_ACTUALIZAR_TTPN_BOOKING_TIPO_SERVICIO.php`	`PATCH /api/v1/ttpn_bookings/:id`	PATCH
`Gasto_ACTUALIZAR_TTPN_BOOKING_COMENTARIO.php`	`PATCH /api/v1/ttpn_bookings/:id`	PATCH
`Gasto_ACTUALIZAR_TTPN_BOOKING_SIN_CHOFER.php`	`PATCH /api/v1/ttpn_bookings/:id`	PATCH
`Gasto_SELECT_TTPN_BOOKINS_FOR_VEHICLE.php`	`GET /api/v1/ttpn_bookings?vehicle_id=X&fecha=Y`	GET

Campos esperados en respuesta (app los usa para mostrar viaje del día)

```
{
  "fecha": "2026-04-10",
  "hora": "08:30:00",
  "id": 1,
  "descripcion": "...",
  "status": true,
  "client_id": 2,
  "clv": "DEV-V001",
  "nombre": "Carlos",
  "apaterno": "Mendoza",
  "servicio": "entrada",
  "cliente": "CLT-001",
  "tipo_servicio": 1,
  "ttpn_service_id": 5,
  "destino": "Chihuahua",
  "cantidad_pasajeros": 10,
  "lat_persona": 28.635,
  "lng_persona": -106.088,
  "lat_planta": 28.640,
  "lng_planta": -106.050
}
```

Nota: `"servicio"` es un campo calculado:
`CASE WHEN ttpn_service_type_id = 1 THEN 'entrada' ELSE 'salida' END`

Los campos `lat_persona`/`lng_persona` vienen de `ttpn_booking_passengers` y `lat_planta`/`lng_planta` de `client_branch_offices`.

---

4. VEHICLE ASIGNATIONS

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_SELECT_VEHICLE_ASIGNATIONS.php`	`GET /api/v1/vehicle_asignations`	GET
`Gasto_INSERT_VEHICLE_ASIGNATION.php`	`POST /api/v1/vehicle_asignations`	POST
`Gasto_ACTUALIZAR_VEHICLE_ASIGNATIONS_FECHA_HASTA.php`	`PATCH /api/v1/vehicle_asignations/:id`	PATCH
`Gasto_SELECT_HISTORIAL_VEHICLE_ASIGNATIONS.php`	`GET /api/v1/vehicle_asignations?historial=true`	GET
`Gasto_SELECT_EXISTENCIA_ASIGNACION.php`	`GET /api/v1/vehicle_asignations/exists?vehicle_id=X&fecha=Y`	GET

Lógica de negocio al crear asignación

- Antes de crear, cerrar asignaciones previas del mismo vehículo (`fecha_hasta = hoy`)
- Después de crear, actualizar `ttpn_bookings` futuros del vehículo con el nuevo `employee_id`

---

5. GAS CHARGES (Cargas de Gas)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_INSERTAR_POST.php`	`POST /api/v1/gas_charges`	POST
`Gasto_INSERTAR_CARGA_GASOLINA.php`	`POST /api/v1/gasoline_charges`	POST
`Gasto_SELECT_CARGAS_CAMIONETAS.php`	`GET /api/v1/gas_charges`	GET
`Gasto_SELECT_CARGAS_CAMIONETAS_GASOLINA.php`	`GET /api/v1/gasoline_charges`	GET
`Gasto_SELECT_GASOLIN_STATION.php`	`GET /api/v1/gas_stations`	GET
`Gasto_ELIMINAR_CARGA_GASOLINA.php`	`DELETE /api/v1/gasoline_charges/:id`	DELETE

⚠ Bug conocido en Rails

El callback `before_create :verificar_id` en `GasCharge` y `GasolineCharge` falla con `NoMethodError` cuando la tabla está vacía (`GasCharge.last` devuelve nil).

Fix pendiente en `ttpngas/app/models/gas_charge.rb:61`:

```
def verificar_id
  ActiveRecord::Base.connection.reset_pk_sequence!('gas_charges')
  actual_id = GasCharge.last
  return unless actual_id  # ← agregar este guard
  next_id = actual_id.id + 1
  # ...
end
```

Aplicar el mismo fix en `gasoline_charge.rb`.

Campos que envía la app

```
{
  "vehicle_id": 5,
  "monto": 1200.00,
  "cantidad": 45.5,
  "odometro": 35000,
  "fecha": "2026-04-10",
  "hora": "10:00:00",
  "lat": 28.635,
  "lng": -106.088,
  "imei": "123456789",
  "ticket": 9001,
  "gas_station_id": 3,
  "created_by": "antonio.castellanos@ttpn.com.mx"
}
```

---

6. VEHICLE CHECKS (Revisiones)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_INSERT_VEHICLE_CHECKS.php`	`POST /api/v1/vehicle_checks`	POST
`Gasto_SELECT_VEHICLE_CHECKS.php`	`GET /api/v1/vehicle_checks`	GET
`Gasto_ACTUALIZAR_VEHICLE_CHECKS_CHOFER.php`	`PATCH /api/v1/vehicle_checks/:id/chofer`	PATCH
`Gasto_ACTUALIZAR_VEHICLE_CHECKS_ENTREGA.php`	`PATCH /api/v1/vehicle_checks/:id/entrega`	PATCH
`Gasto_ACTUALIZAR_VEHICLE_CHECKS_RECEPCION.php`	`PATCH /api/v1/vehicle_checks/:id/recepcion`	PATCH
`Gasto_ACTUALIZAR_VEHICLE_CHECKS_AUDITORIA.php`	`PATCH /api/v1/vehicle_checks/:id/auditoria`	PATCH
`Gasto_SELECT_PUNTOS_REVISION.php`	`GET /api/v1/review_points`	GET

Campos del modelo (vehicle_checks)

```
id, vehicle_id, puntos_originales (INT), fecha_origen (DATE),
puntos_revisados (INT), fecha_revision (DATE),
puntos_auditados (INT), fecha_auditoria (DATE),
puntos_recibidos (INT), fecha_recepcion (DATE),
status (BOOLEAN), historial (TEXT)
```

Verificar: Que la tabla `vehicle_checks` tenga todos estos campos en el schema.

---

7. EMPLOYEES (Empleados)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_SELECT_EMPLOYEES.php`	`GET /api/v1/employees`	GET
`Gasto_SELECT_EMPLOYEE.php`	`GET /api/v1/employees/:id`	GET
`Gasto_ACTUALIZAR_TELEFONO_EMPLOYEE.php`	`PATCH /api/v1/employees/:id`	PATCH
`Gasto_ACTUALIZAR_EMPLOYEE_IMEI.php`	`PATCH /api/v1/employees/:id`	PATCH
`Gasto_ACTUALIZAR_TOKEN_FIREBASE.php`	`POST /api/v1/employees/:id/firebase_token`	POST
`Gasto_ACTUALIZAR_APP_VERSION_EMPLOYEE.php`	`POST /api/v1/employees/:id/app_version`	POST
`Gasto_SELECT_PERFIL_CHOFER.php`	`GET /api/v1/employees/:id/perfil`	GET
`Gasto_SELECT_CLIENT_EMPLOYEES.php`	`GET /api/v1/client_employees`	GET

⚠ Firebase token en campo `password`

El PHP guarda el Firebase token en `employees.password`. **No replicar este patrón**.

Acción: Agregar columna `firebase_token` a la tabla `employees`:

```
rails g migration AddFirebaseTokenToEmployees firebase_token:string
```

El endpoint `PATCH /api/v1/employees/:id/firebase_token` actualiza solo ese campo.

Campos en respuesta (la app usa estos para mostrar el perfil del chofer)

```
{
  "id": 10,
  "nombre": "Carlos",
  "apaterno": "Mendoza",
  "amaterno": "Ríos",
  "clv": "DEV-E001",
  "direccion": "Calle 1 #10",
  "ciudad": "Chihuahua",
  "telefono": "6141234567",
  "puesto": "Chofer",
  "Licencia_Vigencia": "2029-01-01",
  "RFC": "MERC800101AAA",
  "CURP": "MERC800101HCHRNR00",
  "NSS": "123456789",
  "Cuenta_bancaria": "...",
  "key": "s3-key",
  "licence_key": "s3-key-licencia"
}
```

Los campos de documentos (`RFC`, `CURP`, `NSS`, `Licencia_Vigencia`, `Cuenta_bancaria`) vienen de `employee_documents` según el tipo:

- tipo 2 → Licencia de conducir (vigencia)
- tipo 7 → RFC
- tipo 8 → CURP

- tipo 9 → NSS
- tipo 13 → Teléfono
- tipo 14 → Cuenta de banco

---

8. DRIVER REQUESTS (Solicitudes de Viaje)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_INSERT_DRIVER_REQUESTS.php`	`POST /api/v1/driver_requests`	POST
`Gasto_SELECT_DRIVER_REQUESTS_PENDIENTES.php`	`GET /api/v1/driver_requests?status=pendiente`	GET
`Gasto_SELECT_DRIVER_REQUESTS_ACEPTADOS.php`	`GET /api/v1/driver_requests?status=aceptado`	GET
`Gasto_SELECT_DRIVER_REQUESTS_RECHAZADOS.php`	`GET /api/v1/driver_requests?status=rechazado`	GET
`Gasto_SELECT_DRIVER_REQUESTS_*_POR_EMPLEADO.php`	`GET /api/v1/driver_requests?employee_id=X&status=Y`	GET
`Gasto_ACTUALIZAR_STATUS_DRIVER_REQUEST.php`	`PATCH /api/v1/driver_requests/:id`	PATCH

---

9. DISCREPANCIES (Descuadres)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_INSERT_DISCREPANCIES.php`	Interno — creado por callback del modelo	—
`Gasto_SELECT_DISCREPANCIAS_POR_PLANTA.php`	`GET /api/v1/discrepancies?grupo=planta`	GET
`Gasto_SELECT_REPORTE_DESCUADRES.php`	`GET /api/v1/discrepancies`	GET
`Gasto_SELECT_REPORTE_DE_DESCUADRES_POR_CHOFER.php`	`GET /api/v1/discrepancies?grupo=chofer`	GET
`Gasto_ACTUALIZAR_DISCREPANCIA_PNC_CAPTURADO.php`	`PATCH /api/v1/discrepancies/:id`	PATCH

Tipos de KPI (usados en la tabla discrepancies)

- `pnc` — Pasajero no capturado (booking sin travel_count)
- `pcr` — PCR (revenue discrepancy)
- `cnp` — CNP (cost discrepancy)

---

10. COORDINATION (Coordinadores)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_SELECT_COO_TRAVEL_REQUEST.php`	`GET /api/v1/coo_travel_requests`	GET
`Gasto_INSERT_COO_TRAVEL_REQUEST.php`	`POST /api/v1/coo_travel_requests`	POST
`Gasto_SELECT_COO_TRAVEL_EMPLOYEE_REQUEST.php`	`GET /api/v1/coo_travel_employee_requests?coo_travel_request_id=X`	GET
`Gasto_INSERT_COO_TRAVEL_EMPLOYEE_REQUEST.php`	`POST /api/v1/coo_travel_employee_requests`	POST
`Gasto_ACTULIZAR_COO_TRAVEL_EMPLOYEE_REQUEST_COORDINADOR.php`	`PATCH /api/v1/coo_travel_employee_requests/:id`	PATCH

---

11. SCHEDULES / ROUTING (Rutas y Horarios)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_INSERT_CRD_HRS.php`	`POST /api/v1/crd_hrs`	POST

PHP endpoint	Rails endpoint	Método
`Gasto_SELECT_CRD_HRS.php`	`GET /api/v1/crd_hrs`	GET
`Gasto_INSERT_CRDH_ROUTES.php`	`POST /api/v1/crdh_routes`	POST
`Gasto_INSERT_CRDHR_POINTS.php`	`POST /api/v1/crdhr_points`	POST
`Gasto_INSERT_CR_DAYS.php`	`POST /api/v1/cr_days`	POST
`Gasto_SELECT_CHOFERES_VIAJES_POR_PLANTA.php`	`GET /api/v1/travel_counts/por_planta`	GET

Verificar: Si las tablas `crd_hrs`, `crdh_routes`, `crdhr_points`, `cr_days` existen en el schema de Rails.

---

12. CLIENTS (Clientes y Sucursales)

Endpoints PHP → Rails

PHP endpoint	Rails endpoint	Método
`Gasto_SELECT_CLIENTES.php`	`GET /api/v1/clients`	GET
`Gasto_SELECT_CLIENT_BRANCH_OFICCE_POR_CLIENTE.php`	`GET /api/v1/client_branch_offices?client_id=X`	GET
`Gasto_SELECT_CLIENT_BRANCH_OFFICES_LAT_LNG.php`	`GET /api/v1/client_branch_offices?fields=lat,lng`	GET

---

13. MISCELLANEOUS

Endpoints que la app usa y necesitan soporte BE

PHP endpoint	Rails endpoint	Prioridad
`Gasto_SELECT_VIGENCIA_LICENCIAS.php`	`GET /api/v1/employees/licencias_vencimiento`	Alta
`Gasto_SELECT_VERSION.php`	`GET /api/v1/versions/current`	Alta
`Gasto_SELECT_CONEXION_STATUS.php`	`GET /api/v1/employees/conexion_status`	Media
`Gasto_SELECT_TIME.php`	`GET /api/v1/server_time`	Baja
`Gasto_SELECT_CUMPLEANIOS.php`	`GET /api/v1/employees/cumpleaños`	Baja
`Gasto_SELECT_CANTIDAD_VIAJES_CHOFERES.php`	`GET /api/v1/travel_counts/stats`	Media
`Gasto_SELECT_PLANTAS_COORDENADAS.php`	Mismo que `client_branch_offices` con lat/lng	Alta
`Gasto_SELECT_BROAD_CAST_RECEIVER.php`	`GET /api/v1/broadcasts`	Media

---

14. FORMATO DE RESPUESTA — Compatibilidad con la app

La app espera respuestas en este formato del PHP:

```
{ "respuesta": "200", "estado": "OK", "data": [...], "error": null }
```

El Rails API usa formato diferente (HTTP status codes reales). **La app Android necesita actualizarse** para consumir el formato REST estándar, o el BE agrega un adapter.

Recomendación: Actualizar la app para consumir el formato Rails — es el cambio correcto.

---

15. PROBLEMA CRÍTICO — ID generado en cliente

El app Android genera el `id` del nuevo registro haciendo:

```
// Fetch max ID
GET Gasto_SELECT_MAX_TRAVEL_COUNTS.php  →  { max_id: 1234 }
// Insert con id = 1234 + 1
POST Gasto_INSERT_TRAVEL_COUNTS.php  →  { id: 1235, ... }
```

Hay 30+ endpoints `SELECT_MAX_*` para esto.

Rails no debe replicar este patrón — usa autoincrement.
La app debe actualizarse para NO enviar `id` en el body y leer el `id` de la respuesta del POST.

PRIORIDAD DE IMPLEMENTACIÓN

🔴 Inmediato (la app no funciona sin esto)

1. **Auth JWT** — ✅ Ya existe, verificar compatibilidad de respuesta
2. **Travel Counts CRUD** + lógica de booking matching
3. **TTPN Bookings** — vista del día del chofer con lat/lng
4. **Employees** — listado con documentos incluidos (RFC, CURP, etc.)
5. **Client Branch Offices** — con lat/lng (GPS de la app depende de esto)
6. **Fix bug GasCharge `verificar_id`** — nil check en callback

🟡 Necesario para flujo completo

7. **Gas Charges** — POST/GET
8. **Vehicle Asignations** — con lógica de cierre de asignaciones previas
9. **Vehicle Checks** — todos los estados (entrega, chofer, recepción, auditoría)
10. **Driver Requests** — pendientes/aceptados/rechazados por empleado
11. **Firebase Token** — migración a columna dedicada + endpoint PATCH

🔵 Complementario

12. **Discrepancies** — reportes por planta y por chofer
13. **CRD/Routes/Points** — scheduling avanzado
14. **Broadcasts** — notificaciones
15. **Server time** — sincronización
16. **Stats endpoints** — cantidad de viajes por chofer, por planta

---

ESTADO REAL DEL BE — Tablas vs Controladores

Todas las tablas existen en `db/schema.rb`. El Rails schema está adelante del PHP.
El trabajo pendiente es exclusivamente de **controladores y acciones faltantes**.

Controllers que ya existen y cubren la app ✅

Tabla / Módulo	Controller	Acciones disponibles
`travel_counts`	✅	index, show, create, update, destroy, import
`ttpn_bookings`	✅	index, show, create, update, destroy, stats, import
`vehicle_asignations`	✅	index, show, create, update, destroy, finalize, vehicle_history
`gas_charges`	✅	index, show, create, update, destroy, stats
`gasoline_charges`	✅	index, show, create, update, destroy, import, stats
`gas_stations`	✅	—
`vehicle_checks`	✅	index, show, create, update, destroy
`employees`	✅	index, show, create, update, destroy, activate, deactivate
`driver_requests`	✅	—
`service_appointments`	✅	—
`employee_appointments`	✅	—
`employees_incidences`	✅	—
`discrepancies`	✅	—
`clients`	✅	—
`client_branch_offices`	✅	—
`vehicles`	✅	—
`ttpn_services`	✅	—
`ttpn_service_types`	✅	—

Tabla / Módulo	Controller	Acciones disponibles
`ttpn_foreign_destinies`	✅	—
`review_points`	✅	—
`versions`	✅	—

Controllers que FALTAN (tabla existe, no hay controller) ❌

Tabla	Usado en app Android	Prioridad
`coo_travel_requests`	✅ Sí — coordinadores crean solicitudes masivas	🔴 Alta
`coo_travel_employee_requests`	✅ Sí — asignación chofer a solicitud	🔴 Alta
`crd_hrs`	✅ Sí — horarios de choferes	🟡 Media
`crdh_routes`	✅ Sí — rutas dentro del horario	🟡 Media
`crdhr_points`	✅ Sí — waypoints de ruta	🟡 Media
`cr_days`	✅ Sí — días activos del horario	🟡 Media
`broadcast_receivers`	✅ Sí — mensajes broadcast a choferes	🟡 Media
`client_employees`	✅ Sí — empleados asignados a clientes	🔴 Alta
`fixed_routes`	Parcialmente	🔵 Baja

Acciones faltantes en controllers existentes

Controller	Acción faltante	Equivalente PHP
`employees`	`firebase_token` (PATCH)	`Gasto_ACTUALIZAR_TOKEN_FIREBASE.php`
`employees`	`app_version` (PATCH)	`Gasto_ACTUALIZAR_APP_VERSION_EMPLOYEE.php`
`employees`	`perfil` (GET — con documentos embebidos)	`Gasto_SELECT_PERFIL_CHOFER.php`
`travel_counts`	`autorizar` (PATCH)	`Gasto_ACTUALIZAR_TRAVEL_COUNT_AUTORIZADO.php`
`travel_counts`	`rechazar` (PATCH)	`Gasto_ACTUALIZAR_TRAVEL_COUNT_RECHAZADO.php`
`travel_counts`	`comentarios` (POST)	`Gasto_INSERT_TRAVEL_COUNTS_COMENTARIO_POR_ADMIN.php`
`vehicle_checks`	`chofer` (PATCH)	`Gasto_ACTUALIZAR_VEHICLE_CHECKS_CHOFER.php`
`vehicle_checks`	`entrega` (PATCH)	`Gasto_ACTUALIZAR_VEHICLE_CHECKS_ENTREGA.php`
`vehicle_checks`	`recepcion` (PATCH)	`Gasto_ACTUALIZAR_VEHICLE_CHECKS_RECEPCION.php`
`vehicle_checks`	`auditoria` (PATCH)	`Gasto_ACTUALIZAR_VEHICLE_CHECKS_AUDITORIA.php`

NOTAS FINALES

- **Firebase keys** están hardcodeadas en el adapter Android — moverlas a config del servidor
- **Timezones**: El PHP no maneja TZ explícitamente. Rails debe usar UTC y dejar que el FE/app convierta
- **Soft delete**: Algunas tablas usan `status BOOLEAN`, otras `desactivacion VARCHAR` — estandarizar a `discarded_at` o `status`
- **Odómetro**: La app actualiza odómetro en `vehicles` al registrar gas charge — replicar en el callback del modelo